

Tech Challenge - Fase 1

Oficina Mecânica

Documento de Entrega

Pós Tech - 17SOAT - Software Architecture

Data: 31/08/2026

Identificação do grupo

Campo	Informação
Nome do grupo	Grupo 110
Disciplina / Fase	Arquitetura de Software - Fase 1

Participantes

Nome	Username Discord
Derek	Derek rm375085

1. Links de entrega

Item	Link
Repositório GitHub (privado, acesso soat-architecture)	https://github.com/derektsc/Oficina-Tech-Challenge
Documentação DDD (Miro)	https://miro.com/app/board/uXjVHsDM-SM=?share_link_id=386665499309
Vídeo demonstrativo (até 15 min)	https://youtu.be/RyrMkk2pN3Q
Swagger / API Docs (local)	http://localhost:3000/api-docs

2. Resumo do projeto

Este projeto entrega o MVP do back-end de um Sistema Integrado de Atendimento e Execução de Serviços para uma oficina mecânica. A solução é um monolito em Ruby on Rails 8 com PostgreSQL, organizado em camadas com regras de domínio explícitas (DDD). O foco está na gestão de ordens de serviço (OS), clientes, veículos, catálogo de serviços e peças/insumos com controle de estoque.

3. Entregáveis da Fase 1 - checklist

- Back-end monolítico com arquitetura em camadas
- APIs RESTful documentadas via Swagger (/api-docs)
- Dockerfile e docker-compose.yml para execução local
- Autenticação JWT nas rotas administrativas
- Validação de CPF/CNPJ e placa de veículo
- Fluxo completo da OS com 6 status e transições automáticas
- API pública para o cliente acompanhar e aprovar/rejeitar orçamento
- CRUD de clientes, veículos, serviços e peças com estoque
- Métrica de tempo médio de execução dos serviços
- Testes automatizados (RSpec) com cobertura acima de 80% nos domínios críticos
- README.md com instruções de uso e justificativa do banco de dados
- Documentação DDD MIRO (Event Storming, linguagem ubíqua, diagramas)
- Relatório de análise de vulnerabilidades (Brakeman)
- Vídeo demonstrativo de até 15 minutos

4. Stack técnica

Componente	Escolha / versão
Linguagem	Ruby 3.3.6
Framework	Ruby on Rails 8.0
Banco de dados	PostgreSQL 16
Autenticação	JWT + bcrypt
Documentação API	OpenAPI / Swagger (rswag)
Testes	RSpec + FactoryBot + SimpleCov
Segurança	Brakeman + bundler-audit
Containerização	Docker + Docker Compose

5. Justificativa do PostgreSQL

O domínio da oficina é fortemente relacional: um cliente possui veículos; a ordem de serviço (OS) referencia cliente, veículo e itens; peças possuem estoque controlado. O cenário de negócio exige histórico persistente, unicidade de identificadores (CPF/CNPJ, placa, SKU, número da OS) e consistência no débito de estoque quando o cliente aprova o orçamento - situação em que duas aprovações simultâneas não podem consumir o mesmo saldo.

PostgreSQL foi escolhido porque garante integridade referencial (foreign keys), transações ACID e lock de linha no débito de estoque, além de índices únicos e tipos DECIMAL para valores monetários sem erro de arredondamento. Alternativas como SQLite foram descartadas pela limitação de concorrência em escrita; MongoDB/NoSQL não se adequam ao modelo relacional e à consistência exigida no estoque; Redis não substitui persistência transacional do histórico da OS.

Para o MVP em monolito Rails, PostgreSQL oferece maturidade, suporte nativo no ecossistema (migrations, Docker, CI) e caminho de evolução (relatórios, réplicas, JSONB) sem troca de banco.

6. Relatório de análise de vulnerabilidades

Ferramentas utilizadas:

- Brakeman 8.0.6 - análise estática de segurança em aplicações Rails
- bundler-audit - verificação de CVEs nas dependências Ruby

Como reproduzir:

```
docker compose run --rm --no-deps --entrypoint bash app -c "bundle exec brakeman -q"
```

```
docker compose run --rm --no-deps --entrypoint bash app -c "bundle exec bundle-audit check --update"
```

Resultado do Brakeman

Métrica	Valor
Rails	8.0.5.1
Controllers analisados	10
Models analisados	8
Erros de análise	0
Security Warnings	0

Controles de segurança implementados

- JWT (Bearer) em todas as rotas /api/v1/admin/*
- Senhas armazenadas com bcrypt (has_secure_password)
- Validação rigorosa de CPF/CNPJ e placa no domínio
- Token público UUID imprevisível para consulta da OS pelo cliente
- Filtro de parâmetros sensíveis nos logs
- API-only - sem cookies de sessão
- SQL via ActiveRecord - sem interpolação manual